UCS645

PARALLEL & DISTRIBUTED COMPUTING

ASSIGNMENT 4:

Submitted by: KETANPREET SINGH (102303386)

Submitted to: Dr. Saif Nalband

Ring Communication

1. Objective

To implement a parallel MPI program that demonstrates inter-process communication using a ring topology. The experiment evaluates message passing, process coordination, and correctness of data transfer across processes.

2. Problem Description

In a ring communication pattern, each process sends a message to the next process in sequence.

- Process 0 starts with an initial value of 100
- Each process adds its rank to the received value
- The message is passed along the ring
- The last process sends the final value back to Process 0

The goal is to ensure correct cyclic communication and data propagation across all processes.

3. Methodology

1. Initialized MPI environment using MPI_Init()
2. Obtained process rank and total number of processes
3. Process 0 initialized the value to 100 and sent it to Process 1
4. Each process:
   Received value from previous process
   Added its rank
   Sent updated value to next process
5. Used modulo operation: to maintain ring topology
6. Final value returned to Process 0

4. Performance Results

Processes	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000098	1.00	100.0%
2	0.000120	0.82	41.0%
4	0.000220	0.45	11.2%
8	0.000900	0.11	1.3%

5. Performance Discussion

The results show that as the number of processes increases, the **execution time (Tp)** increases instead of decreasing. This leads to a **decline in speedup (Sp)** and **efficiency (Ep)**.This behavior indicates that the parallel implementation suffers from high **overhead**, such as communication, synchronization, and process management costs. Instead of improving performance, adding more processes introduces extra delays.As a result, the system demonstrates **poor scalability**, where increasing computational resources does not lead to better performance. This typically occurs when the problem size is small or the workload is not efficiently distributed among processes.

6. Conclusion

The program successfully demonstrates ring communication using MPI. Correct data propagation across processes confirms proper implementation of point-to-point communication

Parallel Array Sum

1. Objective

To compute the sum of an array using parallel processing in MPI and evaluate collective communication using MPI_Scatter and MPI_Reduce.

2. Problem Description

An array of size 100 is initialized with values from 1 to 100. The array is divided among processes, and each process computes a local sum. The global sum is obtained using reduction.

3. Methodology

1. Process 0 initialized array (1–100)
2. Distributed data using MPI_Scatter()
3. Each process computed local sum
4. Combined results using MPI_Reduce()
5. Computed average value

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000028	1.00	100.0%
2	0.000018	1.56	78.0%
4	0.000012	2.33	58.3%
8	0.000010	2.80	35.0%

5. Performance Discussion

The performance results indicate that increasing the number of processes initially improves execution speed, as seen by the decrease in **execution time (Tp)** and increase in **speedup (Sp)** from 1 to 4 processes. This shows that the system benefits from parallelization in the early stages.

However, as the number of processes increases further (e.g., 8 processes), the improvement becomes less significant or may even degrade slightly. This is reflected in the reduced growth of speedup and a noticeable drop in **efficiency (Ep)**.

The decline in efficiency occurs due to **parallel overheads**, such as communication between processes, synchronization delays, and increased management cost. These factors limit scalability and prevent achieving ideal linear speedup.

Overall, the system demonstrates **moderate scalability**, where performance improves up to a certain point, after which overhead dominates and reduces efficiency

6. Conclusion

MPI_Scatter and MPI_Reduce efficiently distribute and combine data. The program demonstrates effective parallel aggregation with correct results.

Maximum and Minimum

1. Objective

To compute global maximum and minimum values across processes using MPI reduction operations.

2. Problem Description

Each process generates 10 random numbers (0–1000). Each process computes local maximum and minimum. Global values are determined using reduction operations.

3. Methodology

1. Each process generated random numbers
2. Computed local max and min
3. Used:
    - MPI_MAXLOC for global max
    - MPI_MINLOC for global min
4. Stored value + rank using structure

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000030	1.00	100.0%
2	0.000020	1.50	75.0%
4	0.000014	2.14	53.5%
8	0.000017	1.76	22.0%

Performance Discussion

The performance results show that increasing the number of processes improves execution time initially. From 1 to 4 processes, the **execution time decreases** and **speedup increases**, indicating effective utilization of parallelism.

However, when the number of processes increases to 8, the execution time slightly increases and speedup decreases. This indicates that the system starts experiencing **parallel overhead**, such as communication delays, synchronization cost, and process management overhead.

The **efficiency continuously decreases** as the number of processes increases, which is expected in real-world parallel systems due to non-parallelizable portions of the program and overhead factors.

Overall, the system demonstrates **good scalability up to a certain limit (4 processes)**, after which the performance degrades due to overhead dominating the benefits of parallel execution

5. Conclusion

The program demonstrates MPI reduction with location tracking. Efficient computation of global extrema across distributed processes was achieved

Parallel Dot Product

1. Objective
To compute the dot product of two vectors using MPI-based parallel processing.

2. Problem Description
Two vectors:

- A = [1,2,3,4,5,6,7,8]
- B = [8,7,6,5,4,3,2,1]

Vectors are distributed across processes. Each process computes partial dot product, and results are combined.

3. Methodology

1. Distributed vectors using MPI_Scatter()
2. Each process computed partial dot product
3. Used MPI_Reduce() to combine results
4. Final result computed at Process 0

4. Performance Results

Processes (p)	Time (Tp) sec	Speedup (Sp)	Efficiency (Ep)
1	0.000160	1.00	100.0%
2	0.000100	1.60	80.0%
4	0.000072	2.22	55.5%
8	0.000110	1.45	18.1%

5. Performance Discussion

The results show that increasing the number of processes initially improves performance. From 1 to 4 processes, the **execution time (Tp)** decreases significantly, leading to an increase in **speedup (Sp)**. This indicates effective utilization of parallelism and better distribution of workload.

However, when the number of processes increases to 8, the execution time rises and speedup decreases. This suggests that the system is affected by **parallel overheads** such as communication delays, synchronization, and process management costs, which start to outweigh the benefits of parallel execution.

The **efficiency (Ep)** also decreases as the number of processes increases, reflecting reduced resource utilization due to these overheads and possible load imbalance.

Overall, the system shows **good performance up to 4 processes**, after which scalability is limited and performance begins to degrade.

6. Conclusion

MPI_Scatter and MPI_Reduce efficiently distribute and combine data. The program demonstrates effective parallel aggregation with correct results.